

PROBLEM 4

Fraud Detection

Identifying deceptive or malicious transactions in real time

ML Interview Preparation Guide

What Is Fraud Detection?

When your phone buzzes with Did you just spend 4,500 dollars at a jewelry store in Moscow? — how did your bank know within seconds? Fraud detection means automatically identifying transactions, accounts, or actions that are deceptive or malicious, in real time.

Real World Examples: Credit card fraud, account takeover, return fraud, bot fraud draining ad budgets, insurance fraud, app install fraud.

Step 0: Clarify the Problem

- **Type of fraud?** Card fraud? Account takeover? Both?
- **Real-time or batch?** Credit card must be real-time under 300ms. Insurance claims can be batch.
- **False positive vs false negative cost?** Missed fraud = bank loses money. Blocking legitimate customer = angry customer, support costs.
- **Who are the adversaries?** Fraudsters actively adapt. Fraud is an adversarial problem — your model improves, fraudsters change tactics.

Step 1: Understanding the Data

The Extreme Class Imbalance Problem

Fraud is extremely rare — typically 0.1% to 2% of all transactions. A model that always says not fraud gets 99.8% accuracy but catches zero fraud. Accuracy is a terrible metric for fraud.

Fraud Pattern Analysis

Amount: Legitimate = mostly 5-200 dollars. Fraud = bimodal: very small (1 dollar test transactions) or very large (3000+ dollars). Time: Fraud peaks at 3-5am when users are asleep. Merchant category: High fraud = Electronics, Jewelry, Gift Cards, Wire Transfers.

Velocity Patterns — The Strongest Signal

Velocity = how fast events happen. Normal: Coffee shop 8am, Restaurant noon, Grocery 6pm. Fraudster: Amazon 200 dollars at 3:01am, Electronics 800 at 3:03am, Gift card 500 at 3:05am, Wire transfer 2000 at 3:07am. Fraudsters drain accounts as fast as possible before the card is blocked.

Step 2: Feature Engineering

Transaction-Level Features

- `transaction_amount`, `merchant_category`, `country`, `hour_of_day`
- `is_international` — transaction country differs from user home country

User Historical Baseline Features

Fraud is anomalous relative to the user's own history. A 5000 dollar transaction is normal for a wealthy user but suspicious for someone who usually spends 20 dollars.

- `amount_vs_user_avg_30d` — this transaction vs user avg over 30 days
- `amount_vs_user_max_ever` — is this larger than anything they have ever spent?
- `is_new_merchant` — user never shopped here before?

Velocity Features

- `num_transactions_last_1h` — how many in last hour?
- `num_unique_countries_last_24h` — multiple countries in one day?
- `num_declined_transactions_last_1h` — failed attempts = card testing

The Impossible Travel Feature

TX_001: New York, 2:00pm. TX_002: London, 2:45pm. 45 minutes, 5570 km apart — physically impossible. Strong fraud signal.

Device and Network Features

- `is_new_device` — user never used this device before?
- `is_tor_or_vpn` — suspicious anonymizing network?
- `num_accounts_on_this_device` — one device used by many accounts = fraud ring

Graph Features (Advanced)

Fraud rings are networks of connected accounts, devices, and merchants. If Account_A is confirmed fraud and shares Device_X with Account_B, Account_B is suspicious. Graph features: `num_fraud_accounts_sharing_same_device`, `degree_of_separation_from_known_fraud_node`.

Step 3: Statistics and Math

Bayes Theorem

$P(\text{Fraud} \mid \text{Features}) = P(\text{Features} \mid \text{Fraud}) \times P(\text{Fraud}) / P(\text{Features})$. Even with a 50x suspicious signal, if fraud is 0.2% base rate, the posterior is only about 9%. This is why thresholds must be set carefully from business costs.

Expected Value Framework

If we ALLOW: legit = 0 cost; fraud = -500 dollars loss. If we BLOCK: legit = -30 dollars (angry customer); fraud = 0. Block if: $P(\text{fraud}) > 30/530 = 5.7\%$. Compute threshold from business costs, not arbitrarily.

Step 4: Models

Baseline: Rule-Based System

Always start here before ML. Block if: international transaction for domestic-only user, amount 10x larger than user average, 5+ transactions in last hour, known bad IP. Pros: fast, interpretable, regulator-friendly. Cons: fraudsters adapt. Used alongside ML, not replaced by it.

Logistic Regression

$P(\text{fraud}) = \text{sigmoid}(w_1 \times \text{amount} + w_2 \times \text{is_new_country} + w_3 \times \text{velocity} + \text{bias})$. Interpretable coefficients make it easy to explain decisions to regulators. Example: $\text{is_new_country weight} = +2.3$ (strong fraud signal), $\text{familiar_merchant} = -1.5$ (reduces probability).

XGBoost / LightGBM — Industry Standard

Gradient boosting: sequential trees where each corrects mistakes of previous ones. Set $\text{scale_pos_weight} = \text{num_legitimate} / \text{num_fraud}$ to handle class imbalance. SHAP values give per-transaction explanations essential for compliance.

Handling Class Imbalance

- **SMOTE**: Create synthetic fraud examples by interpolating between real fraud cases. Only apply to training data, never test.
- **Undersampling**: Reduce legitimate examples to balance dataset.
- **Ensemble approach**: Train 50 models on different random undersamples. Average predictions.

Deep Learning Models

LSTM on transaction sequences: use raw sequence of events per user. LSTM builds a representation of spending pattern. A new transaction that does not fit scores high for fraud.

Graph Neural Networks (GNN): passes fraud signals across edges in the fraud ring network. After 2 rounds of message passing, each account representation includes info from accounts 2 hops away. Detects fraud rings that individual transaction models miss completely.

Step 5: Evaluation

The Right Metrics

- **Precision:** Of all fraud alerts, how many were actually fraud? (False alarm rate)
- **Recall:** Of all actual frauds, how many did we catch? (Miss rate)
- **F1 Score:** $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
- **AUC-PR:** Precision-Recall curve — better than ROC-AUC for highly imbalanced data

Threshold Strategies

Different thresholds for different transaction amounts. High-value (over 5000 dollars): lower threshold, be aggressive. Low-value: higher threshold, be conservative. Compute break-even threshold from expected value framework.

Step 6: Production

Real-Time Scoring — 300ms Hard Constraint

1. Fetch user features from Redis feature store — 10ms
2. Compute real-time velocity features via sliding window — 20ms
3. Run rule-based checks — 5ms
4. Run ML model — 15ms
5. Combine scores and apply threshold — 5ms

Total: ~60ms with 240ms buffer. Redis operations take microseconds — essential for velocity feature serving at scale.

Three-Tier Decision System

- **Score 0.0 to 0.3: AUTO APPROVE** — transaction goes through instantly
- **Score 0.3 to 0.7: STEP-UP AUTH** — verify with OTP or biometric
- **Score 0.7 to 1.0: AUTO DECLINE** — transaction blocked, customer notified

The Adversarial Problem

Fraudsters adapt. Month 1: model catches jewelry store fraud at night. Month 2: fraudsters switch to electronics during business hours. Continuous retraining is essential.

- **Frequent retraining:** Weekly or daily on new confirmed fraud cases.
- **Concept drift detection:** Monitor fraud detection rate daily. Drop triggers retraining.
- **Investigator feedback loop:** Investigator verdicts become new training labels.

Explainability for Compliance

Banks must often legally explain why transactions were declined. SHAP values give per-transaction explanations. Customer message: Your transaction was declined because it was in a new country and the amount is significantly higher than your usual spending.

How to Answer in an Interview

Fraud detection is a real-time, adversarial, class-imbalanced classification problem. Clarify the latency constraint and the cost of false positives vs. false negatives — this determines the threshold strategy. Key features: velocity features over sliding time windows, user-relative amount features, and the impossible travel feature. Build a layered system: rule-based checks first, then LightGBM for complex patterns, then GNN for fraud rings. Handle class imbalance with scale_pos_weight and SMOTE. Evaluate on AUC-PR and F1. Use expected value framework for threshold. Redis for real-time feature serving. SHAP for explainability. Retrain weekly on investigator-confirmed labels. Monitor concept drift daily.